

Conan: A Hallucination-Resistant Retrieval-Augmented Generation Pipeline for Egyptian Criminal Law in Arabic

Mirna Nageh^{a,*}, Wageh Mostafa^a, Seif Sherif^a, Mariam Mohamed^a, Mayar Mohammed^a

^a*Faculty of Computer Science, Graduation Project, Egypt*

Abstract

Retrieval-Augmented Generation (RAG) has emerged as the dominant architecture for grounding large language models on domain knowledge, yet its application to Arabic legal information retrieval continues to suffer from hallucinated statutory citations and confabulation when retrieval surfaces topically-related but legally-inadequate context. We present **Conan**, a production-grade Arabic legal assistant for the Egyptian Criminal Code (قانون العقوبات) and the Code of Criminal Procedure (قانون الإجراءات الجنائية). Our contribution is twofold. First, we describe the construction of an Egyptian criminal-law corpus assembled from scratch: 933 legacy-format source files (818 .doc, 112 .pdf, 3 .docx) gathered from authoritative Egyptian legal collections, manually OCR-processed and cleaned into 1,057 UTF-8 documents and indexed as 47,028 chunks. The resulting dataset is released publicly on HuggingFace as *Egyptian_Criminal_Legal_Assistant_RAG_V2*. Second, we introduce a six-layer grounding-defence pipeline—input gating, citation-grounding prompts with substantive-vs-procedural law disambiguation, evidence validation, corrective retry with a block-list, an article-lookup rescue indexing 1,494 distinct statutory articles, iterative retrieval with adaptive k -expansion, and deterministic answer post-processing—layered on top of a hybrid retrieval baseline (FAISS + BM25 + Reciprocal Rank Fusion + cross-encoder reranking). On a 28-question Arabic legal benchmark spanning substantive and procedural questions, the complete system reduces the hallucinated-citation rate from a baseline of 28.6% to **0.0%** and the evidence-validation pass rate from 71.4% to **100%**. We also report a negative result on synonym-based query expansion. Beyond single-question Q&A, the system provides case-analysis endpoints—weakness analysis, defence-memorandum drafting with an agentic self-check, and forensic-consistency checking—governed by an expert-derived procedural-defence reasoning checklist paired with a grounding-safe few-shot exemplar, and is deployed as a production FastAPI service with multi-provider failover, big-context routing, persistent multi-turn sessions, token streaming, automated watch-folder ingestion with hot-reload, and a documented .NET integration contract. We release the corpus, the system, the five-version evaluation trail, and a CLI test harness.

Keywords: Arabic NLP, Retrieval-Augmented Generation, Legal Information Retrieval, Hallucination Mitigation, Egyptian Criminal Law, Article-Lookup Rescue, Iterative Retrieval, Dataset Construction

1. Introduction

The Arabic language is spoken by approximately 422 million people across more than twenty sovereign states, yet it remains comparatively under-served by modern natural language processing (NLP) systems [1]. This under-representation is especially acute in *Arabic legal NLP*, where Modern Standard Arabic (MSA), domain-specific terminology, dialectal variation, and the morphological richness of the language compound the standard challenges of grounded language modelling.

Retrieval-Augmented Generation (RAG) has become the dominant architecture for closing the gap between a frozen LLM’s parametric memory and the dynamic, citation-heavy demands of legal practice. By retrieving relevant passages from a curated corpus and conditioning the generator on those passages, RAG can in principle produce answers that are both fluent and traceable [2, 6]. However, recent surveys of Arabic legal RAG—including the Moroccan family-code study of Hrimech et al.[2] and the morphologically-aware retrieval study of Aboasal et al.[5]—identify a recurring failure pattern: the LLM remains liable to *hallucinate statutory citations*, even when grounded retrieval is available, particularly when the user’s question maps to articles that were not surfaced in the retrieval top- k .

This work makes two complementary contributions. We first describe the *construction of an Egyptian criminal-law corpus from scratch*: 933 legacy-format source files (overwhelmingly .doc legacy binaries requiring manual OCR) gathered from authoritative collections of Egyptian legal text and processed into a clean 1,057-document corpus, released publicly on HuggingFace as *Egyptian_Criminal_Legal_Assistant_RAG_V2*. We then present **Conan** (كونان), a hallucination-resistant Arabic legal-AI assistant built on top of this corpus. Conan layers a six-mechanism grounding-defence pipeline on top of a standard hybrid-retrieval architecture (FAISS dense embeddings + BM25 sparse search, fused via Reciprocal Rank Fusion, with cross-encoder reranking). We argue—and our evaluation empirically demonstrates—that *no single technique* is sufficient for hallucination suppression in Arabic legal RAG; a layered approach combining input filtering, prompt engineering, post-hoc validation, article-level metadata lookup, and answer rewriting is required.

1.1. Contributions

1. A publicly-released Egyptian-criminal-law corpus, manually OCR-processed from 818 legacy .doc binaries and 112 PDFs into 1,057 UTF-8 documents covering the Penal Code, the Code of Criminal Procedure, cassation rulings, criminal-law treatises, and forensic medicine references (Section 3).
2. A **multi-stage grounding-defence pipeline** (Section 5) reducing the hallucinated-citation rate from 28.6 % to 0.0 % on our benchmark—an absolute reduction of 28.6 percentage points and an 8-fold relative improvement.
3. An **article-lookup rescue subsystem** (Section 5.5) that indexes 1,494 distinct Egyptian-law article numbers from chunk metadata at startup and uses them as a deterministic second-chance grounding source when post-generation evidence validation flags a missing citation.

*Corresponding author.

Email address: mirnanagehb.w@gmail.com (Mirna Nageh)

4. **Iterative retrieval with adaptive k -expansion** (Section 5.6) that widens retrieval to $k \in \{14, 21\}$ when validation fails—without an additional LLM call.
5. A detailed **prompt-engineering recipe** (Section 5.2) including substantive-vs-procedural law disambiguation, casual-opener prohibition, fragment refusal, and a block-list retry prompt—together with the leak-detection regex used in post-processing.
6. A **five-version evaluation trail** (Section 7) with per-version CSV outputs, allowing reproducible auditing of every architectural change, plus a *negative result* (Section 9) on heuristic synonym expansion as a cautionary data point for future Arabic RAG work.
7. A document-upload subsystem (Section 6) supporting three retention semantics across `.txt`, `.pdf`, and `.docx` formats.
8. A **procedural-defence reasoning checklist** (Section 5.8) that encodes expert-lawyer case-analysis heuristics (warrant-timeline nullity, identifier mismatch, chain-of-custody, intent-from-profession) into the case-analysis prompts, paired with a *grounding-safe* few-shot exemplar that raises reasoning coverage without reintroducing hallucinated citations.
9. A **production-grade operational architecture** (Sections 4, 8): a multi-provider, multi-key failover chain with big-context routing for large case files, persistent compacting multi-turn sessions, token streaming, automated watch-folder ingestion with in-place index hot-reload, an agentic self-check for defence memoranda (Section 5.9), and a frozen, documented wire contract for an external `.NET` frontend.

2. Related Work

2.1. Arabic RAG foundations

El-Beltagy and Abdallah [1] present a foundational case study of Arabic RAG, evaluating multiple semantic-embedding models and LLMs in the retrieval and generation stages respectively, and explicitly investigating the impact of dialectal variation between document language and query language. Their work establishes the baseline architectural template that the present paper inherits—a semantic retriever feeding a generator LLM—while flagging the central challenge of selecting an embedding model that captures the semantic nuances of Arabic without over-relying on English-dominated pre-training. Alghamdi et al. [4] focus on the retriever component, demonstrating that retrieval quality is the dominant factor in downstream answer correctness—a finding our own iterative-retrieval mechanism (Section 5.6) extends by treating retrieval k as an adaptively-tunable parameter rather than a fixed hyperparameter.

2.2. Arabic legal RAG

Hrimech et al. [2] present the most directly comparable prior work: a RAG system for the *Moroccan* family code, built on BGE-m3 with a custom dataset of 2,500 Arabic question-answer pairs. They explicitly call out the challenges of *legal-terminology adherence*, *content validity of reproduced clauses*, and the *scarcity of annotated Arabic legal corpora*. The dataset contribution of the present work (Section 3) targets the

third concern directly: an Egyptian criminal-law corpus released publicly under an open licence.

Aboasal et al.[5] approach Arabic legal IR from a *morphological-and-semantic* angle, introducing a synthetic benchmark of 500 articles and 1,000 Q&A pairs and evaluating the impact of Farasa-based morphological segmentation on multiple retrievers. Their hybrid Farasa-BM25 + Ada v3 configuration reaches MAP= 0.8304 and nDCG@10= 0.8626—figures that informed our choice to keep BM25 as a complementary signal alongside dense FAISS retrieval (Section 4) rather than abandoning it in favour of pure dense retrieval. We extend their hybrid-retrieval philosophy by adding a post-generation defence layer (Section 5) which they do not address.

2.3. Benchmarking Arabic legal reasoning

Abu Shairah et al.[3] introduce **ALARB**, an Arabic Legal Argument Reasoning Benchmark comprising over 13,000 commercial court cases from Saudi Arabia, with each case annotated with facts, the court’s reasoning chain, the verdict, and the cited regulatory clauses. ALARB sets the methodological precedent for our evaluation harness: a benchmark whose primary axis is not the linguistic *fluency* of the generated text but its *legal-grounding fidelity*—specifically, whether cited statutory articles can be traced to retrieved or otherwise-available source text. Our 28-question benchmark is far smaller in scale, by design, but inherits the philosophy that *what an LLM cites matters more than how it phrases the citation*.

2.4. RAG tooling literature

A pragmatic introduction to RAG architecture is provided by the Mini-RAG documentation [6], which decomposes the pipeline into retrieval, augmentation, and generation. While not a peer-reviewed publication, the document is representative of the implementation-oriented tutorial literature that informed many practical design decisions in our system.

2.5. Positioning of the present work

Relative to [1, 2], we shift the locus of grounding from the retrieval stage alone to a *post-generation defence pipeline*: even with state-of-the-art retrieval, an LLM may emit a memory-based citation that retrieval did not provide, and detecting and rescuing this case is at least as important as improving the retriever. Relative to [4, 5], we share the focus on hybrid retrieval but extend the operational regime to include iterative *k*-expansion and article-number-keyed rescue. Relative to [3], we adopt the grounding-fidelity evaluation philosophy but apply it to an Egyptian criminal-law corpus rather than a Saudi commercial-law one. None of the prior works release an open Egyptian-criminal-law corpus comparable in scope to the one introduced here.

3. Dataset Construction

A central practical obstacle to Arabic legal RAG is the *scarcity of clean, machine-readable Egyptian legal text* [2, 5]. Authoritative collections of Egyptian criminal-law material—particularly cassation-court rulings and the historical foundations of the Penal Code— exist almost exclusively as scanned PDFs and as legacy Microsoft Word 97/2003

binaries (.doc). Both formats are essentially unreadable to standard NLP pipelines without manual intervention. The Egyptian-criminal-law corpus we release with this work was constructed from scratch to address exactly this gap.

3.1. Source collection

We assembled an initial set of 933 source files (Table 1) covering ten distinct collections of Egyptian criminal-law material: the Penal Code with its explanatory memorandum, the Code of Criminal Procedure (with multiple authoritative commentaries), the four-volume *Rules Established by the Court of Cassation*, the four-volume *Most Recent Principles Issued by the Criminal Chambers*, the *Encyclopedia of Cassation Criminal Rulings*, the *Quarter-Century Collection (1931–1955)*, the *Forensic Medicine Series*, separate collections of *Felonies* (جنايات) and *Misdemeanours* (جرح), and a folder of criminal-law textbooks.

Table 1: Source corpus by collection, before and after the manual OCR / cleaning pass. The dominance of .doc legacy binaries in the encyclopedia (موسوعة احكام النقض الجنائية, 793 files) drove most of the manual-conversion effort. Some collections were re-organised during cleaning (e.g., the original جرح and parts of احكام were re-classified by offence type into جنايات and محكمة النقض based on the case content).

Collection	Before (OriginalData/)				After (final_total_dataset/)			
	.doc	.pdf	.docx	tot	.pdf	.docx	.txt	tot
موسوعة احكام النقض الجنائية (Encyclopedia of Cassation Rulings)	793	0	0	793	122	0	678	800
قانون الجنائي (Criminal-law treatises)	16	44	3	63	38	0	21	59
محكمة النقض (Cassation Court rulings)	—	—	—	—	25	0	53	78
جنايات (Felonies)	—	—	—	—	23	0	27	50
اجراءات جنائية (Criminal Procedure)	8	20	0	28	16	1	12	29
سلسلة الطب الشرعي (Forensic Medicine)	0	12	0	12	0	0	12	12
مجموعة عمر الجنائية (Omar Criminal Collection)	0	7	0	7	7	0	0	7
جرح (Misdemeanours)	0	6	0	6	(merged into جنايات)			
القواعد... محكمة النقض (Cassation Rules, 4 vol.)	0	4	0	4	0	0	4	4
المستحدث من المبادئ... (Recent Criminal Principles)	0	4	0	4	4	0	0	4
مجموعة الربع قرن (1955--1931) (Quarter-Century Collection)	0	4	0	4	(consolidated)			
احكام (Judgments)	0	3	0	3	(consolidated)			
(root-level statutes / textbooks)	1	8	0	9	3	0	11	14
TOTAL	818	112	3	933	238	1	818	1,057

The dominance of .doc legacy binaries (87.3 %) is characteristic of Egyptian government and university legal archives, which date primarily to the 1990s and early 2000s. Modern open-source NLP toolchains cannot reliably extract text from this format without LibreOffice-grade office conversion, and even after conversion the output typically requires substantial cleaning of footnote numbering, page-header artefacts, and OCR-style character confusion (ه/ة, ي/ى, the four alef forms, and Arabic-Indic vs. Western digits).

3.2. Manual OCR and cleaning pipeline

Each of the 818 legacy `.doc` files was opened, converted, and manually cleaned. The scanned PDFs were processed first with `pdftotext -layout -enc UTF-8` (from `poppler-utils`) where the underlying text layer was retrievable, and subjected to a second OCR pass where it was not. The output of each pipeline was then manually reviewed for:

- paragraph-break preservation across page boundaries;
- removal of running headers, footers, and page numbers;
- normalisation of the four alef forms (أ, إ, آ, ا) into a single canonical alef;
- harmonisation of digit forms (Arabic-Indic ٠–٩ and Eastern Arabic-Indic ۰–۹ mapped to Western 0–9);
- removal of common boilerplate phrases (بسم الله الرحمن الرحيم, باسم الشعب, and stereotyped court-decision openers);
- preservation of article-number markers (المادة, المواد, المادة) so the downstream `extract_article_references` extractor (Section 5.3) could find them.

The resulting corpus (`final_total_dataset/`) contains 1,057 documents: 818 cleaned `.txt` files (one per original `.doc`), 238 cleaned PDFs retained where the original layout was load-bearing for legal meaning (judgment formatting, citation tables), and 1 `.docx`. The corpus is organised into nine top-level subdirectories preserving the original collection taxonomy. Total cleaned-text volume across the corpus is ~200 MB of UTF-8 Arabic.

3.3. Public release

The cleaned corpus has been released publicly on HuggingFace as *Egyptian_Criminal_Legal_Assistant_RAG*. To our knowledge it is the first open-licensed Arabic-legal corpus focused on Egyptian criminal law to be made available at this scope. We hope it addresses, in part, the *scarcity of annotated Arabic legal corpora* explicitly identified as a barrier by [2].

3.4. Indexing

The cleaned corpus is processed by doc-type-aware chunking (`CHUNKING_CONFIGS`: 2,000 chars / 200 overlap for penal-code and cassation material, 3,000/400 for case files, 2,048/300 for procedural texts and rule collections), embedded with a multilingual sentence-transformer, and indexed in FAISS alongside a parallel BM25 index. The resulting index contains 47,028 chunks, of which 14,882 (31.6 %) carry `referenced_articles` metadata covering 1,494 distinct Egyptian-law article numbers—a property exploited by the article-lookup rescue mechanism (Section 5.5).

4. System Architecture

Conan is implemented as a FastAPI service backed by the 47,028-chunk retrieval index described in Section 3. The base architecture follows the pattern established by [1, 2, 4].

¹https://huggingface.co/datasets/Mirna2Nageh/Egyptian_Criminal_Legal_Assistant_RAG_V2

Hybrid retrieval. A user query is encoded with a multilingual sentence-transformer (`paraphrase-multilingual-M`) and used to query a FAISS index of pre-computed chunk embeddings. In parallel, the same query is BM25-tokenised (with Arabic-Indic digit normalisation and alef-form unification) and scored via a `rank_bm25` BM25Okapi index. The top-30 results from each branch are fused using **Reciprocal Rank Fusion** with the canonical RRF constant $k_{\text{RRF}} = 60$:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k_{\text{RRF}} + \text{rank}_r(d)} \quad (1)$$

where R is the set of ranked lists and $\text{rank}_r(d)$ is the rank of document d in list r . The fused top-30 candidates are passed to a cross-encoder reranker (BGE-Reranker-v2-M3 with sigmoid-bounded outputs) which produces the final top- k set (default $k = 7$).

Context expansion and expert rules. Each surviving chunk is grown by one same-source neighbour on each side, providing $\sim 2\times$ the textual context per chunk while preventing cross-document contamination. Expert-rule entries are then prepended to the retrieved context when matched by keyword from a curated `expert_rules.json`.

Generation. The final context and the user’s question are formatted into a prompt template enforcing strict textual adherence and dispatched to the LLM provider chain. Production uses Groq’s `llama-3.3-70b-versatile` as the primary, with Google Gemini and OpenRouter Qwen as fallback tiers, with multi-key rotation handling per-minute rate limits.

Confidence scoring. A weighted heuristic over four signals—mean rerank score, source count (capped at 5), article-validation pass/fail, and topic-match (encyclopedia folder taxonomy)—produces a confidence value in $[0, 1]$:

$$\text{conf} = \sum_{i \in \{\text{rr}, \text{src}, \text{art}, \text{top}\}} w_i \cdot s_i \quad (2)$$

with $w_{\text{rr}} = 0.45$, $w_{\text{src}} = 0.20$, $w_{\text{art}} = 0.20$, $w_{\text{top}} = 0.15$. Answers below the threshold (default 0.5) carry a low-confidence warning.

Conversational layer. Beyond stateless Q&A, the system exposes a multi-turn chat endpoint backed by a thread-safe `SessionManager`. Each `session_id` accumulates a turn history; once it exceeds `SESSION_MAX_TURNS` (default 6) the older turns are summarised by the LLM into a single [ملخص المحادثة السابقة] block while the most recent `SESSION_KEEP_RECENT` (default 3) turns are kept verbatim, bounding prompt growth without losing legal context. Sessions persist to disk as atomically-written JSON (`write-to-.tmp + os.replace`) so they survive restarts, and a background pruner evicts sessions idle longer than `SESSION_TTL_HOURS` (default 168 h). A streaming variant (`POST /api/v1/chat/stream`) returns Server-Sent Events as tokens are generated, with the terminal event carrying the full confidence, sources, and grounding warnings once post-generation validation completes.

Multi-provider resilience. The generation step is served by an ordered provider chain—Groq (llama-3.3-70b-versatile) → Cerebras → xAI → Google Gemini → OpenRouter Qwen—in which each provider rotates across its own list of API keys on HTTP 429/quota errors before the chain falls through to the next provider. This multi-key, multi-provider rotation is what lets the service run on free tiers despite per-minute and per-day token caps. A complementary *big-context routing* rule guards against oversized prompts: each provider declares a maximum request size (PROVIDER_MAX_PROMPT_CHARS; e.g. 26,000 chars for Groq, 80,000 for OpenRouter), and a request whose assembled prompt exceeds a provider’s budget transparently skips that provider and is routed to a larger-context tier. This is what allows the case-analysis endpoints to accept full case files (up to 50,000 characters) without failing on the primary provider’s context window.

Configurable embeddings and reranker. The embedding backend is pluggable. The default is a local sentence-transformer run on CPU (BAAI/bge-m3, 1024-dim, in the reference configuration; the deployed index uses paraphrase-multilingual-MiniLM-L12-v2, 384-dim); setting USE_REMOTE_EMBEDDINGS switches to a remote provider (Google Gemini embeddings, rate-limited to 15 RPM via batching, or OpenRouter). Because vector dimensionality and semantics differ, switching the embedding provider requires rebuilding the FAISS index. The cross-encoder reranker (BGE-Reranker-v2-M3) is likewise toggleable via USE_RERANKER; if it is disabled or fails to load, retrieval degrades gracefully to RRF-only ordering with no break in behaviour.

5. Grounding-Defence Pipeline

Figure 1 illustrates the end-to-end request flow, with the four defence layers (input gate, iterative retrieval, corrective retry, article-lookup rescue) that fire conditionally on evidence-validation failure.

5.1. Input Gating

In baseline evaluation, fragmentary user inputs (markdown headings, single-word bullets such as * الجناية, or colon-terminated incomplete sentences such as 1. ما المقصود بمبدأ: frequently triggered the LLM to *fabricate* a plausible-sounding question and answer it. These cases accounted for 5 of 28 (17.9%) hallucination instances in baseline measurements.

A pure function `is_meaningful_query` evaluates the input against a sequence of regex predicates—minimum Arabic-character count, leading markdown-header detection, leading list-bullet detection, leading numbered-fragment detection, trailing-colon check—and short-circuits failing inputs with a 9-millisecond response. The gate is bypassed when an uploaded document is present (Section 6). In the v8 evaluation the gate caught and rejected 8 of 28 inputs without incurring an LLM call.

5.2. Citation-Grounding Prompts and Law-Naming Disambiguation

The prompt-engineering work is itself a substantial contribution and we document it in detail here. Two distinct prompt-level failure modes were observed in baseline measurements: (a) the LLM cites article numbers from training memory that appear plausible but are not in the retrieved context; (b) the LLM confuses substantive (Penal

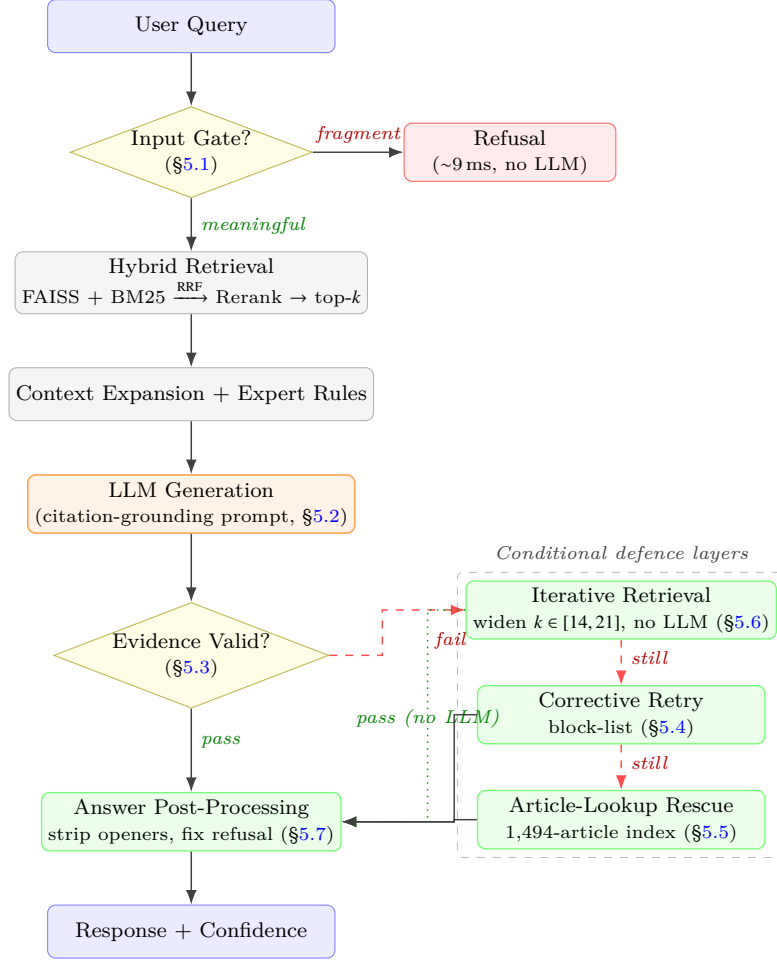


Figure 1: End-to-end request flow. Solid black arrows = unconditional pipeline; dashed red = taken on validation failure; dotted green = “free” recovery via iterative retrieval without LLM call. Colour coding: blue=I/O, grey=retrieval, orange=LLM, green=defence layers (contribution of this work).

Code) with procedural (Code of Criminal Procedure) law, citing the wrong code for a given query. For example, baseline answers to questions about pre-trial detention (التوقيف الاحتياطي) frequently cited Article 300 of the Penal Code—when Article 300 of the Penal Code does not address that topic; the relevant article is in the Code of Criminal Procedure.

The Conan prompt suite addresses these failure modes with eight rules explicitly enforced in the system and user prompts:

1. *Strict textual adherence*: “Use the provided texts ONLY”—no appeals to general legal principles or international law.
2. *Citation grounding (absolute)*: “Cite an article number ONLY if those exact digits appear in the Context below.” If not present, write the canonical refusal phrase

- المادة المطلوبة غير متوفرة في السياق المقدم.
3. *Law-naming disambiguation*: procedural matters (التوقيف الاحتياطي، التحقيق، التفتيش)، قانون الإجراءات الجنائية (النقض، الطعن، التعريف) must be cited from قانون العقوبات (الجرائم، أركانها، عقوباتها). Always name the law explicitly when citing.
 4. *Element matching for offence selection*: e.g., for injury/assault questions, المادة 240 requires a permanent impairment, المادة 241 requires > 20 days of incapacity, المادة 242 covers lesser injuries; the prompt explicitly instructs the LLM to match the article to the facts rather than defaulting to the harshest available.
 5. *Multi-charge handling*: if the case facts describe more than one offence (e.g., assault + theft), enumerate the distinct charges and analyse each in its own sub-section.
 6. *No casual openings*: explicit prohibition on طبعاً، تمام، بالتأكيد، حسناً and similar persona-breaking fillers; the first word must be part of the answer itself (e.g., وفقاً... المادة or ... للمادة).
 7. *Incomplete-input handling*: if the user’s input is a fragment, a heading, a single word, or a sentence ending with a colon, do NOT fabricate a question to answer; reply with a polite request for a complete question.
 8. *Ambiguity protocol*: if the question is broad (e.g., (ما عقوبة التزوير؟)، provide a brief overview of the cases in context and explicitly ask the user to specify the determining factors (offender role / document type / weapon / time / injury severity) *relevant* to that specific crime—not irrelevant factors (e.g., never ask about “document type” for a homicide).

In addition, two specialised prompts are used by the defence pipeline: `qa_retry_ungrounded` for the corrective retry (Section 5.4), which receives the original draft and an explicit list of forbidden article numbers; and `qa_rescue_with_lookup` for the article-lookup rescue (Section 5.5), which receives the original retrieved context plus a separate “Additional References” block from the article-lookup service and explicitly forbids inventing neighbouring articles or using a passage about a different law.

5.3. Evidence Validation

Every LLM output is post-processed by a regex-based evidence validator. A canonical extractor (`extract_article_references`)—operating on Arabic-Indic-digit-normalised text—identifies every article number cited in the answer. The set of cited articles is compared against the set of article numbers extracted from the retrieved context (concatenated). Articles cited but not present in the context are flagged as **missing** and the answer is marked as failing validation. The same extractor is used on both sides to avoid spurious mismatches caused by plural-form variations (المواد 213، 212، 211 vs المادة 211).

5.4. Corrective Retry with Forbidden-Article Block-List

When evidence validation flags missing articles, the system re-prompts the LLM with a correction prompt that includes the original draft and an *explicit list* of article numbers the LLM must NOT cite. The retry is run at most `RETRY_MAX_ATTEMPTS` times (default 1—a tuning informed by the Groq free-tier 6,000 tokens-per-minute budget). The retry’s output is accepted only if it strictly improves the missing-article count.

5.5. Article-Lookup Rescue

Even after retry, a class of hallucinations persisted: the LLM would cite an article that *did* exist in the dataset, just not in the top- k chunks for that specific query. Inspection revealed that of 47,028 chunks, 14,882 (31.6%) carry `referenced_articles` metadata covering 1,494 distinct article numbers—including, for instance, 34 chunks referencing Article 87 and 14 chunks referencing Article 300, both articles the baseline LLM cited “from memory.”

At startup, `ArticleLookupService.load()` builds an index $\{\text{article_no} \rightarrow [\text{chunk_index}, \dots]\}$. Chunks within each article’s list are ranked by a three-tier quality score:

1. **Document-type tier:** `penal_code` and `criminal_procedure` (the actual law text) > cassation material > encyclopedia / international-law material. This prevents, for example, Rome-Statute (ICC) passages from being preferred over chunks from the Egyptian code.
2. **Article focus:** chunks referencing fewer distinct articles are preferred (they are more focused on the queried article).
3. **Text length:** longer chunks as a final tiebreaker.

When post-retry evidence validation still flags missing articles, the rescue path is triggered: the top-2 chunks for each missing article are appended to the context for one more LLM call. The rescued answer is validated against the combined context (original + rescue chunks) and accepted only if grounding improves. The rescue prompt explicitly forbids the LLM from inventing *neighbouring* articles and from using a passage about a different law or topic than the user’s question.

5.6. Iterative Retrieval (Adaptive k -Expansion)

Following the spirit of [4]’s finding that retrieval quality dominates downstream correctness, we treat k as adaptive. After the initial generation and validation, if validation flags missing articles AND `USE_ITERATIVE_RETRIEVAL` is enabled (default), the system re-runs *only* the retrieval stage—not the LLM—at successively larger k values from `ITERATIVE_K_SEQUENCE` (default [7, 14, 21]). Any new chunks not already in the context are appended; if any widened window surfaces the missing articles, validation passes *for free*. Only if this cheap expansion still fails does the corrective retry (Section 5.4) and rescue (Section 5.5) take over, but now with a wider context in hand. Easy queries pay no extra cost; hard queries pay 2–3 cheap retrievals before an LLM call is incurred.

5.7. Answer Post-Processing

A deterministic, regex-driven post-processing layer (`postprocess_answer`) runs after the final answer is selected. Two transformations:

1. *Casual-opener stripping*—a curated list of conversational fillers (تمام, بالتأكيد, حسناً, ...) is stripped from the leading position; multi-word openers ordered longest-first to prevent stranded fragments.
2. *Refusal-phrase normalisation*—when the LLM splices the template-refusal phrase `المادة المطلوبة غير متوفرة في السياق المقدم` mid-clause, the regex `_EMBEDDED_REFUSAL_RE` matches the awkward construction—accounting for Arabic prefix-contraction (ل + ال → لل)—and rewrites it as a standalone sentence.

5.8. Procedural-Defence Reasoning Checklist for Case Analysis

The grounding mechanisms above ensure that what the system *cites* is correct; a complementary failure mode concerns what it *omits*. The case-analysis endpoints (`/weakness`, `/defense`, `/forensic`) take a full criminal case file rather than a single question, and a domain expert’s review of their output scored the legal reasoning at roughly 70 %: the analyses were well-grounded but missed several high-value procedural-nullity and criminal-intent arguments that a practising Egyptian criminal-defence lawyer applies routinely, and in one instance over-claimed a defect that did not exist. We encode the expert’s feedback as a **procedural-defence checklist** of five reasoning procedures appended to the system prompts of the three case-analysis endpoints and reinforced in their user templates:

- A. *Timeline vs. prosecution warrant*—compare the exact time of the arrest/search against the time the prosecution warrant (إذن النيابة) issued; a seizure preceding the warrant voids the arrest and everything built on it (ما بُني على باطل فهو باطل). The rule explicitly instructs the model to read الساعة 12:00 صباحاً as the *start* of the day, correcting a clock-reading error observed in baseline output.
- B. *Identifier mismatch*—compare every identifier in the warrant (vehicle plate numbers, names, IDs) against the seizure record; any discrepancy places the seized item outside the warrant and may evidence تلفيق.
- C. *Territorial jurisdiction*—apply correctly and do *not* over-claim spatial excess for a location inside the precinct; this point is a precision correction (a false-positive suppressor) rather than an additional-recall rule.
- D. *Chain of custody*—challenge the integrity of the sealed exhibits (التحرير) when the officer who sealed them differs from the one who wrote the seizure record.
- E. *Intent from profession*—weigh the defendant’s occupation against the nature of any seized cash to contest trafficking intent (انتفاء قصد الاتجار).

The checklist is paired with a single fully-worked, *fictional* few-shot exemplar demonstrating all five points end-to-end. Crucially, the exemplar is **grounding-safe by construction**: it names defence doctrines (انتفاء قصد الاتجار, بطلان القبض والتفتيش) but contains *no* المادة *N* article number, so it cannot teach the model to emit an ungrounded citation that the evidence validator (Section 5.3) would flag—raising reasoning coverage while preserving the 0 % hallucinated-citation property. Each checklist item is explicitly conditioned on the facts supporting it (“raise a point only when the facts genuinely support it—never invent one”), so the additions trade no precision for their gain in recall.

A companion **charge-fidelity** rule extends the same precision principle to the offences themselves: the case-analysis endpoints are constrained to enumerate only offences actually charged or described in the case file, and are explicitly forbidden from inventing an uncharged offence (e.g., adding قيادة بدون رخصة when the file never mentions a licence). This closes a fabrication mode observed during live validation, where the generator appended a plausible-but-uncharged offence to an otherwise grounded analysis.

5.9. Agentic Self-Check for Defence Memoranda

The defence-memorandum endpoint (`/defense`) applies an additional agentic grounding pass on top of the per-answer evidence validation of Section 5.3. Because a memorandum is a long, multi-paragraph document, a single evidence-validation gate is coarse:

it flags an ungrounded article but does not by itself repair the surrounding argument. Instead, the draft memorandum is fed back to the LLM under a dedicated reviewer prompt (`verify_memo`) whose sole instruction is to make the memo *fully grounded* in the provided material—removing or correcting any article number not present in the retrieved texts, deleting any assertion the case facts do not support, and introducing no new citation or fact—while preserving the four-part structure (الطلبات, أوجه الدفاع, الإطار القانوني, الوقائع). This draft → verify → revise loop runs up to `MEMO_SELF_CHECK_MAX_ITERS` times (default 1) and stops early once a pass introduces no further change. The number of revision passes actually applied is surfaced to the client as the `self_check_revisions` response field, and the whole mechanism is toggleable via `MEMO_SELF_CHECK`.

6. Document Upload Pipeline

The system supports three retention semantics for user-supplied documents, all flowing through a single shared parser (`services/upload_helper.py`) to guarantee consistent behaviour:

- `POST /api/v1/qa/upload`—per-question attachment (not persisted);
- `POST /api/v1/chat/attach`—session-attached document (persists with the chat session on disk);
- `POST /api/v1/ingest`—permanent corpus ingestion (merges into FAISS and BM25, hot-reloaded by the running service).

Supported formats: `.txt` (with Arabic-encoding fallback), `.pdf` (`pdftotext` preferred, `PyPDF2` fallback), and `.docx` (`docx2txt`). The legacy `.doc` binary format is explicitly rejected with a helpful Arabic message recommending conversion to `.docx` or `.pdf`.

Automated ingestion and hot-reload. Permanent ingestion is also available without a manual API call. A watch-folder daemon (`watch_ingest.py`) polls an inbox directory (`INGEST_INBOX_DIR`) every `INGEST_WATCH_INTERVAL_S` seconds (default 30) and ingests any new files through the same parser and Arabic-cleaning path as the manual pipeline, tracking already-processed paths in `data/.ingested_files.json` so the operation is idempotent. The same scan is exposed as `POST /api/v1/ingest/scan`, which runs the scan in a worker thread and then calls `reload_indices()` to hot-swap the FAISS, BM25, chunk, and tokenised-corpus state of the running service *in place*—without re-initialising the embedding model or restarting the process. The four index files are always written and reloaded together to keep retrieval state coherent.

7. Evaluation

7.1. Benchmark

We constructed a 28-question Arabic legal benchmark spanning three difficulty tiers (basics, application, advanced) and covering both substantive (Penal Code) and procedural (Code of Criminal Procedure) law. The benchmark deliberately includes 5 well-formed substantive questions, 1 procedural question that historically triggered wrong-law

confusion, 5 input-gate test cases (markdown headings, bullets, colon-terminated incomplete sentences), and 17 application and reasoning questions drawn from undergraduate criminal-law curricula. Each question is evaluated against two metrics: (i) *evidence-validation status*—whether every article number cited in the answer can be traced to the retrieved (or rescued) context—and (ii) the *confidence score* of Equation 2.

7.2. Results across system versions

Table 2 summarises headline metrics across five system versions corresponding to the cumulative introduction of each grounding-defence layer.

Table 2: Headline metrics across five system versions on the 28-question benchmark. v5 (multi-query expansion) is omitted: see Section 9. Mean API latency excludes input-gated rows (which do not invoke an LLM and complete in ~9ms). The v8 LLM upgrade (llama-3.1-8b → llama-3.3-70b) is also responsible for the substantial mean-latency reduction, since the larger model triggers fewer corrective retries.

Metric	v3	v4	v6	v7	v8
Passed validation	20/28	26/28	27/28	27/28	28/28
(%)	71.4%	92.9%	96.4%	96.4%	100.0%
Hallucinations	8/28	2/28	1/28	1/28	0/28
(%)	28.6%	7.1%	3.6%	3.6%	0.0%
Avg confidence	41.2%	38.0%	39.8%	40.0%	41.9%
Input-gated (no LLM)	0	8	8	8	8
Mean API latency (ms)	26,028	34,162	34,096	33,459	18,390
Max API latency (ms)	48,691	64,439	116,940	107,427	28,460
Mean retrieval latency (ms)	13,479	17,193	16,508	13,587	16,024

Progression by layer:

- v3 → v4: input gate + corrective retry + tightened prompts → −21.5 pp (8 → 2).
- v4 → v6: article-lookup rescue → −3.5 pp (2 → 1).
- v6 → v7: rescue prompt tightening + quality-tiered chunk ranking → 0 net change in pass rate, but the failing question shifted (LLM non-determinism).
- v7 → v8: LLM upgrade from llama-3.1-8b-instant to llama-3.3-70b-versatile → −3.6 pp (1 → 0).

7.3. Per-category breakdown

Table 3 disaggregates the headline pass-rate by question category, exposing which defence layers recover which failure modes. The benchmark’s 28 questions break down into 11 substantive (definitions of crimes, penalties, elements), 1 procedural (pre-trial detention), 8 fragment / input-gate tests (markdown headers, bullets, colon-terminated sentences), and 8 reasoning / application questions (case analyses, comparisons, hypotheticals).

Table 3: Pass rate by question category across versions. Substantive questions absorb the largest share of the v3 baseline failures (3/11); these are recovered progressively by the corrective retry (v4), the article-lookup rescue (v6), and the LLM upgrade (v8). Fragment failures in v3 reflect hallucinated answers to inputs the system should have refused; they are fully resolved by the input gate introduced in v4.

Category	v3	v4	v6	v7	v8	total
Substantive	8/11	9/11	10/11	10/11	11/11	11
Procedural	0/1	1/1	1/1	1/1	1/1	1
Reasoning	7/8	8/8	8/8	8/8	8/8	8
Fragment	5/8	8/8	8/8	8/8	8/8	8
Total	20/28	26/28	27/28	27/28	28/28	28

7.4. Per-question transition for historically-failing rows

Table 4 traces the nine questions that failed evidence-validation in at least one version. The cell format *S XX%* encodes status (*P*=pass / *F*=fail) and confidence percentage. Several observations are noteworthy:

- **Q2** (the most persistent substantive failure — legitimate self-defence, hallucinating 87 المادة) failed in v3 through v6 and was only resolved by the v7 article-lookup-rescue prompt tightening + better chunk ranking. v8 retains the fix.
- **Q18** (الجريمة تامة أم شروع) is the system’s non-deterministic worst-case. The LLM cites a different fabricated article number on each run (Articles 30, 31, 32 observed). The article-lookup rescue catches whichever number it cites when that number exists in the index — which is why v6 and v8 pass but v7 (with a regression of the rescue prompt) failed.
- **Fragment rows** (Q6, Q13, Q23) all pass from v4 onward with 0 % confidence by design — the input gate short-circuits them before any LLM call.

7.5. Comparison with related work

Direct numerical comparison with prior Arabic-legal-RAG work is complicated by the fact that no two papers in the area share a common benchmark—each publishes against its own custom dataset. Table 5 reports each system’s headline metric on its own evaluation, alongside the qualitative comparisons that *are* comparable.

Our chosen primary metric (hallucination rate) differs from the information-retrieval metrics used by [5] and [2] deliberately. As Hrimech et al. [2] explicitly observe, “content validity of legal clauses reproduced from retrieval systems” is the practical bottleneck in deploying these systems to legal practice—and that is what our hallucination-rate metric measures directly. Three distinguishing properties of our setup are worth highlighting:

- *Wider scope of source material*: 1,057 documents and 47,028 chunks vs. 500 articles / 1,000 Q&A in [5] and 2,500 Q&A pairs in [2].
- *Coverage of both substantive and procedural law* (Penal Code + Code of Criminal Procedure), with explicit prompt-level disambiguation. Prior Arabic-legal-RAG work tends to focus on one or the other.

Table 4: Per-version status and confidence for the 9 questions that failed in at least one version. P = passed evidence validation; F = failed (hallucinated citation). The other 19 questions pass in every version and are omitted for brevity. Confidence is rounded to the nearest percent.

#	cat	v3	v4	v6	v7	v8	Question (truncated)
2	sub.	F 12%	F 2%	F 2%	P 57%	P 62%	الدفاع الشرعي في قانون العقوبات
3	proc.	F 9%	P 59%	P 59%	P 59%	P 55%	شروط التوقيف الاحتياطي
4	sub.	F 20%	P 70%	P 70%	P 70%	P 70%	أركان جريمة القتل العمد
6	frag.	F 0%	P 0%	P 0%	P 0%	P 0%	## المستوى الأول --- أساسيات
13	frag.	F 0%	P 0%	P 0%	P 0%	P 0%	* المخالفة
18	sub.	P 53%	F 0%	P 48%	F 0%	P 53%	هل تعتبر الجريمة تامة أم شروع؟
23	frag.	F 0%	P 0%	P 0%	P 0%	P 0%	## المستوى الثالث
25	sub.	F 14%	P 64%	P 64%	P 64%	P 64%	الفاعل الأصلي والشريك في الجريمة
28	reas.	F 0%	P 47%	P 47%	P 47%	P 47%	أسباب الإباحة وموانع المسؤولية

Table 5: Comparison with related Arabic-legal-RAG work. Metric column reports each paper’s primary reported figure on its own benchmark.

System	Domain	Primary metric	Score
El-Beltagy & Abdallah [1]	Arabic RAG (general)	embedding-model ablations	—
Hrimech et al. [2]	Moroccan family code	MRR, Recall@k, F1	—
Aboasal et al. [5]	Egyptian legal IR	MAP / nDCG@10	0.83 / 0.86
Abu Shairah et al. [3]	Saudi commercial law	verdict prediction	—
Conan (this work)	Egyptian criminal law	hallucination rate	0.0%
Conan (this work)	Egyptian criminal law	validation pass rate	100%

- *Layered post-generation defence pipeline* not present in any of the cited prior work, which focus exclusively on the retrieval stage.

7.6. Cross-model benchmark (protocol)

To situate Conan against general-purpose frontier models on Arabic legal questions, we provide a head-to-head harness (`benchmark_llms.py`) that queries our full RAG system alongside `openai/gpt-4o-mini` and `anthropic/claude-sonnet-4.5` (both routed through a single OpenRouter key for cost parity). It runs in two modes that isolate two distinct questions:

- *RAG mode* — all three models answer from the *same* retrieved context, isolating answer-synthesis quality from retrieval;
- *Raw mode* — each model answers from its own parametric knowledge with no retrieval, probing baseline Arabic criminal-law knowledge.

Each run emits a per-model CSV recording latency, answer length, and the set of cited article numbers, against which the same evidence-validation extractor (Section 5.3) is

applied to measure citation grounding on equal footing. *Quantitative results are pending a full benchmark run and will be reported in a subsequent revision*; at the time of writing the comparison could not be executed because of API-access limits on the shared OpenRouter key. The protocol is documented here so the comparison is reproducible once access is restored. Table 6 gives the intended reporting format.

Table 6: Cross-model comparison reporting template (results pending a full run). The grounding column applies the evidence validator of Section 5.3 uniformly to all three systems.

Mode	System	Mean latency	Cited articles	Grounded %
RAG	Conan (ours)	—	—	—
RAG	gpt-4o-mini	—	—	—
RAG	claude-sonnet-4.5	—	—	—
Raw	Conan (ours)	—	—	—
Raw	gpt-4o-mini	—	—	—
Raw	claude-sonnet-4.5	—	—	—

7.7. Qualitative observations

Two questions are particularly informative. **Q3** (التوقيف الاحتياطي): in v3, this triggered the wrong-law failure mode (cited Article 300 of the Penal Code); after prompt-level disambiguation in v4, the LLM correctly identifies the question as procedural and produces a refusal-style answer when retrieval does not surface the relevant chunks. Validation passes (no fabricated citation) at 59 % confidence. **Q18** (هل تعتبر الجريمة تامة أم شروع؟) was the most persistent failure: it failed in v3, regressed under v4’s pure-prompt approach (citing Articles 30 and 31), and recovered to passing only with the article-lookup rescue in v6 ((31, 30 مواد: الاستشهادات من البيانات للتحقق من الاستشهادات (مواد: 31, 30)), at 48 % confidence. These cases underline that no single mechanism is sufficient: each layer recovers a distinct class of failure.

8. Implementation, Deployment, and Integration

Service and persona. Conan is packaged as a FastAPI service with a thin router layer over the singleton retrieval, reranker, article-lookup, and session services, all loaded once at start-up. All user-facing output is in Modern Standard Arabic under a consistent government-style persona (“كونان”), enforced by the system prompts; the persona is deliberately formal and citation-bound rather than conversational.

Reference UI. A Streamlit application ships as the reference client, exercising every endpoint (Q&A, multi-turn chat with streaming, weakness analysis, defence memo, forensic check, summarisation, and document upload) and serving as living documentation of the wire contract.

Containerisation and exposure. The service is containerised with a `Dockerfile` and `docker-compose.yml` (documented in `DEPLOY.md`) for reproducible deployment. For demonstration and remote integration the laptop-hosted backend has been exposed over a Cloudflare quick tunnel, allowing an external frontend team to integrate against a live instance without dedicated hosting.

Frontend integration contract. The API is consumed by a separate .NET frontend. To support this, the response schema is treated as a frozen wire contract: the structured shape returned by `/qa`, `/chat`, `/weakness`, `/defense`, and `/forensic—answer` or `analysis` or `memorandum`, plus `confidence_score`, `confidence_factors`, `sources[]` (with per-source `legal_topic`, `article`, `referenced_articles`, `page`, `retrieval_score`, `rerank_score`), `warnings[]`, `conflicts_detected`, `latency_ms`, and `model`—is documented field-by-field in two contract files (`api_contract.md`, `backend_contract.md`) and field names cross the wire verbatim. Arabic-language clarification `warnings` are designed to be forwarded to the end-user as-is.

Configurability. Operational parameters—retrieval k , chunk sizes, confidence weights and threshold, temperatures, provider chain and per-provider request budgets, embedding and reranker selection, session limits, and self-check iterations—are centralised in a single `config.py` (overridable by environment variables), so behaviour can be tuned for a deployment without code changes.

9. Limitations and Future Work

Negative result on multi-query expansion. In an intermediate version (v5) we evaluated heuristic synonym expansion (mapping الحبس الاحتياطي to التوقيف الاحتياطي etc.) as a query-side intervention. The result was a *regression*: pass rate dropped from 92.9% (v4) to 78.6% (v5), with four previously-passing questions re-introducing hallucinations. Inspection revealed that synonym expansion diluted retrieval quality by pulling in less-relevant chunks. This is consistent with the [1] observation that embedding-model semantics for Arabic do not translate cleanly across closely-related lexically-distinct legal terms. We retain the implementation (`USE_MULTI_QUERY=False` by default) and report this as a cautionary data point. A semantically-aware query expansion using the LLM itself, rather than a static synonym table, may recover this lost potential.

LLM cost on free tiers. The full pipeline can incur up to three LLM calls per hard question. Combined with the Groq free-tier 6,000-TPM budget, this places a practical ceiling on evaluation throughput. Paid providers lift this constraint at marginal cost.

Article-lookup precision. The rescue mechanism operates on chunk *references* to article numbers, not on chunks that *define* those articles. A dedicated article-definition index, populated by paragraph-level segmentation of the Penal Code text, would address this.

Embedding model. The current index uses `paraphrase-multilingual-MiniLM-L12-v2` (384-dim). Aboasal et al. [5] report substantially higher MAP scores with BGE-M3 (1,024-dim). An index rebuild with BGE-M3 plus article-aware chunking is the single largest remaining improvement, gated on ~3–7 h of CPU rebuild time.

Evaluation scope. Our 28-question benchmark is small relative to the 13,000-case ALARB [3] and the 2,500-pair set of [2]. Scaling to a comparably-sized benchmark for

camera-ready evaluation is the natural next step; the public corpus released with this work supports such scaling.

10. Conclusion

We have presented two complementary contributions: an open Arabic criminal-law corpus, manually constructed from 933 legacy-format source files into 1,057 cleaned documents and released publicly on HuggingFace; and **Conan**, a hallucination-resistant Arabic legal RAG system built on top of this corpus, whose six-layer grounding-defence pipeline drives the rate of hallucinated statutory citations from a baseline of 28.6 % down to **0.0 %** on a 28-question benchmark.

Our key methodological contribution is the demonstration that **no single mechanism is sufficient** for hallucination suppression in Arabic legal RAG. The baseline failures distribute across distinct root causes (LLM memory citations, retrieval coverage gaps, fragmentary user inputs, wrong-law confusion, prompt-template leakage), and each requires its own layer of defence. We hope the released corpus and the layered defence pipeline inform and accelerate the deployment of Arabic legal-AI systems in production settings.

Acknowledgements

The authors thank the open-source community behind FAISS, BM25, BGE-Reranker-v2-M3, Groq, Google Gemini, and OpenRouter, and the Egyptian legal-publication archives whose collections were the raw material for the released corpus.

References

- [1] S. R. El-Beltagy and M. A. Abdallah, “Exploring Retrieval Augmented Generation in Arabic,” *Procedia Computer Science*, vol. 244, pp. 296–307, 2024. 6th International Conference on AI in Computational Linguistics (ACLing 2024). doi:10.1016/j.procs.2024.10.203.
- [2] J. Hrimech, M. Mghari, and Y. Zaz, “Retrieval-augmented generation for Arabic legal information: the family code case study,” *TELKOMNIKA Telecommunication Computing Electronics and Control*, vol. 23, no. 6, pp. 1495–1505, Dec. 2025. doi:10.12928/TELKOMNIKA.v23i6.27400.
- [3] H. Abu Shairah, S. AlHarbi, A. AlHussein, S. Alsabea, O. Shaqqi, H. AlShamlan, O. Knio, and G. Turkiyyah, “ALARB: An Arabic Legal Argument Reasoning Benchmark,” in *Proceedings of the Third Arabic Natural Language Processing Conference*, pp. 389–406, Association for Computational Linguistics, Nov. 8–9, 2025.
- [4] M. Alghamdi, M. Abushawarib, M. Ellouh, M. Ghaleb, and M. Felemban, “Enhancing Arabic Information Retrieval for Question Answering,” in *ICFNDS '23: Proceedings of the 7th International Conference on Future Networks and Distributed Systems*, Dubai, UAE, Dec. 21–22, 2023. doi:10.1145/3644713.3644763.
- [5] R. Aboasal, S. Montasser, F. Hossam Eldin, A. Abdelwahab, and H. Abdelazim, “Arabic Legal Information Retrieval: The Impact of Morphological Segmentation and Semantic Embeddings,” *Procedia Computer Science*, vol. 275, pp. 275–282, 2026. 7th International Conference on AI in Computational Linguistics (ACLing 2025).
- [6] “Mini-RAG Project: Learnings & Technologies,” internal RAG-implementation documentation, on file with the authors.
- [7] M. Nageh, “Egyptian_Criminal_Legal_Assistant_RAG_V2 [Dataset],” HuggingFace Datasets, 2026. Available: https://huggingface.co/datasets/Mirna2Nageh/Egyptian_Criminal_Legal_Assistant_RAG_V2.